

Github Repository Link: <https://github.com/DanielCBruni/csce465-hw2-danielbruni>

Daniel Bruni

10/04/2026

Professor Gu

CSCE 465 - 500

HW2: Protect Agent Messages with Classic Cryptography

Task 0

Task 0 setup involved first running the necessary package install commands and python installation verification commands to ensure that all necessary components were up to date.

Please see the repository at this link: <https://github.com/DanielCBruni/csce465-hw2-danielbruni>

And the screenshot for the necessary command outputs as follows:

```
(.venv) student@student-VirtualBox:~/csce465-agentsec$ mkdir hw2
(.venv) student@student-VirtualBox:~/csce465-agentsec$ ls
hw1  hw2
(.venv) student@student-VirtualBox:~/csce465-agentsec$ cd hw2
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ openssl genpkey -genparam -algorithm DH -pkeyopt group:ffdhe3072 -out ffdhe3072.pem
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ openssl dhparam -in ffdhe3072.pem -text -noout | head -3
DH Parameters: (3072 bit)
GROUP: ffdhe3072
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/student/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/student/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$
```

Github Repository Link: <https://github.com/DanielCBruni/csce465-hw2-danielbruni>

Daniel Bruni

10/04/2026

Professor Gu

CSCE 465 - 500

Task 1

Task 1 began by creating the necessary `baseline_ctr.py` file and populating it with the necessary commands to encrypt, XOR bit flip, and decrypt the plaintext, all the while producing an output at each step. The output of task 1 is as follows:

```
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ python baseline_ctr.py
[*] Original Plaintext: {"action":"READ","path":"notes.txt"}
[*] Ciphertext (hex): 1beffa1e6802d32c77a2f8794928be99eae8b88975f082649e7297eee6c384c6ba0a90a7

--- Demonstrating Ciphertext Malleability (Bit-Flipping) ---
Modified byte at index 11: 'R' -> 'E' using XOR mask 0x17
Modified byte at index 12: 'E' -> 'X' using XOR mask 0x1d
Modified byte at index 13: 'A' -> 'E' using XOR mask 0x04
Modified byte at index 14: 'D' -> 'C' using XOR mask 0x07
[*] Receiver Processed Modified Message: {"action":"EXEC","path":"notes.txt"}

--- Demonstrating Replay Attack ---
Sending original valid ciphertext block 1st time...
-> Receiver successfully processed: {"action":"READ","path":"notes.txt"}
Sending exact same original valid ciphertext block 2nd time (Replay)...
-> Receiver successfully processed again: {"action":"READ","path":"notes.txt"}
[!] Warning: Receiver processed the exact same command twice because there is no counter or timestamp.
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$
```

To analyze what overall happened during this task, first, what design was made must be discussed. For this experiment, the python file in question imitates a basic communication script. The message was encrypted and decrypted using AES-CTR, and was built around the payload of an agent command for reading a `notes.txt` file. Overarchingly, the script uses symmetric encryption with a random key and nonce to encrypt the initial message. Largely, the security of this encryption was incredibly basic, with no external checks or validations for authentication, simply relying entirely on the encryption for confidentiality.

What happened in the experiment was effectively that the script simulated some common attacks on this simplistic form of encryption that took advantage of the lack of authentication checks. Effectively, the first attack was to flip the bits on the encrypted stream, targeting each bit in the word “READ” and using a XOR mask to rebuild the message into “EXEC” without using any sort of key. Thus, when the decryptor activated, it decrypted the bits to finish with “EXEC” as the payload instead of “READ”, a huge issue with message integrity. The other attack was a replay attack where the original ciphertext was sent a second time, thus getting accepted, decrypted, and processed twice without checking for duplicate processes.

The big-picture root cause for why these attacks are so effective on this type of system is that overarchingly, the simple methodology that AES-CTR uses, when combined with a lack of external authentication checks, is simple to bypass. Because the functions act as a stream cipher where the plaintext is XOR’d with some key, a linear operation occurs that is easy to predict,

Github Repository Link: <https://github.com/DanielCBruni/csce465-hw2-danielbruni>

Daniel Bruni

10/04/2026

Professor Gu

CSCE 465 - 500

target, and alter. Without some sort of external check, there is no authentication as to whether or not the initial message payload was altered in some way, as well as no way to check whether or not there are duplicate messages coming in since each message is evaluated independently. Thus, attackers can target, indefinitely and successively, this simple linear transformation with another bit XOR over and over again without the system flagging any sort of integrity issues.

Task 2

Task 2 focused on creating a Diffie-Hellman exchange through the use of the handshake.py python script file. The handshake occurs between a simulated gateway and node pair. Overarchingly, the task requires a few external security implementations that the unauthenticated Diffie-Hellman handshake normally overlooks. For example, the task requires implementing endpoint authentication, such that 3072 bit RSA signing keys are used to bind entities in the exchange itself. Furthermore, a unique and new DH key pair is generated along with new and random nonces for each session. This ensures that no key pair is reused and thus, vulnerable. There were also more security aspects implemented such as role and signature verification for both parties as well as secure session key transmission.

The script itself first loads the parameters from the .pem file created in task 0 of the setup, and gives each entity of the handshake, both gateway and node, independent RSA key pairs. Then, it generates new Diffie-Hellman keys and random nonces for the unique session. Then, it packs all protocol parameters into a byte string and hashes it, thus creating a unique fingerprint for the whole handshake process. Then it has each entity create a signature of their authentication roles on the hash, which effectively validates that both entities are who they are supposed to be and has access to their correct unique private keys and session parameters. Lastly, it computes the DH shared secret and creates all keys for the session based on it. The output of the program is relatively simplistic as follows:

```
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ nano handshake.py
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ python handshake.py
[*] Loading ffdhe3072 group parameters from ffdhe3072.pem...
[*] Starting Authenticated Diffie-Hellman Handshake (Task 2)...
[+] Handshake successfully completed and verified by both parties!
    - Session ID: fbe28bcb7d50e3e4
    - Master Key: 13a43fa1edef1ca77d6a42a8eee327ef...
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$
```

In terms of the overall analysis, there is an explanation for why basic man in the middle attacks and reflection attacks are prevented through this system. Effectively, by signing the transcript mentioned previously on the byte string, you have a sort of key to understanding who the parties are communicating in the handshake. As such, if, for example, an attacker attempts to

Github Repository Link: <https://github.com/DanielCBruni/csce465-hw2-danielbruni>

Daniel Bruni

10/04/2026

Professor Gu

CSCE 465 - 500

intercept some Diffie–Hellman values, the hash of the byte string will change and the system can automatically identify that the transcripts no longer match, thus integrity has been compromised.

For reflection attacks, having the explicit authentication roles within the hash, in this case gateway and node, prevents someone from rerouting a message back to the origin. This is due to the fact that when this would occur, the role prefix will change as to who is the verifier of the message, and the reflected payload will trigger an error within the transcript hash not matching.

Without the signatures mentioned above, the origin authentication previously mentioned is no longer possible. Verifying who the sender is largely relies on the signatures that are stored in the transmission parameters in that byte string hash. Without those signatures, an attacker could easily curate a man in the middle attack by establishing multiple independent connections with the endpoints and then injecting, modifying, or viewing confidential messages without being detected in the signature. Thus, the whole system becomes vulnerable to loss of integrity without any sort of authentication.

Task 3

During task 3, the main goal was to implement a script, `secure_record.py`, that establishes a communication layer for messaging with authenticated encryption. Instead of implementing separate security measures for either integrity or authentication, the design cryptographically binds unique tracking values, message headers, and encrypted payloads under a single verifier shared between the parties. With this form, the receiver can validate the authenticity and precise order of every transmission before allowing any data to be processed within the communication. By implementing these measures, effectively, there is protection against any sort of injections, redirections, alterations, or duplications, which ensures that all communications between the two endpoints maintains both authentication and integrity.

The script encapsulates this sort of structure through the main functions, especially the `seal()` and `open_record()` functions. The `seal` function has the goal of taking the plaintext and associated session parameters and packaging it up to prepare for secure transmission throughout the communication. It does this by creating a unique header, an encryption of the plaintext, an encryption tag, and a few other unique identifier components that get bundled into a byte string.

The `open_record` function is the inverse of this process, largely responsible for validating incoming traffic on the receiver side of the communication. The script does this by running security checks for those unique identifiers discussed previously; checks the integrity of the tag, checks message direction, chronological ordering to prevent repetition attacks, and a few other security checks. Any sort of integrity failures result in the script immediately rejecting the record without ever decrypting any sort of data and terminating. If all checks pass, the script will decrypt the ciphertext and return the corresponding message data.

Github Repository Link: <https://github.com/DanielCBruni/csce465-hw2-danielbruni>

Daniel Bruni

10/04/2026

Professor Gu

CSCE 465 - 500

Task 4

Task 4 revolved around creating 6 test cases for different threats or protocols within the communications mentioned in the previous tasks. These tests included; valid handshake and bidirectional messages, modified ciphertext, modified authenticated header, replayed record, record reflected into the opposite direction, and various authentication failures. The tests were stored in the hw2/tests/test_security.py file and were run using pytest. The output of running the command at the hw2 directory level can be seen below:

```
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2/tests$ cd ..
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$ pytest -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/student/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/student/csce465-agentsec/hw2
collected 6 items

tests/test_security.py::test_valid_handshake_and_bidirectional_messages PASSED [ 16%]
tests/test_security.py::test_modified_ciphertext PASSED [ 33%]
tests/test_security.py::test_modified_authenticated_header PASSED [ 50%]
tests/test_security.py::test_replayed_record PASSED [ 66%]
tests/test_security.py::test_reflected_record PASSED [ 83%]
tests/test_security.py::test_handshake_authentication_failures PASSED [100%]

===== 6 passed in 0.55s =====
(.venv) student@student-VirtualBox:~/csce465-agentsec/hw2$
```

The first test, valid handshake with bidirectional messages, focused on verifying a clean DH communication in both directions between two endpoints. It uses the seal and open_record functions from the previous task to verify the parameters mentioned previously.

The second test, modified ciphertext, tested for payload integrity by immediately deleting and rejecting any sort of injected data or tampered communications. The test accomplishes this by flipping a byte in the ciphertext of a sealed record, and then checks it with the open_record function, raising an error.

The third test, modified authentication header, verifies the integrity of the authentication header that contains vital communication metadata and parameters. It accomplishes this the same way as the second test, but modifying a byte in the header instead of the ciphertext.

The fourth test, replayed record, verifies against duplicating commands or communications by sending the same record multiple times successively. The check occurs when the receiver intakes the same record in succession and processes this against the expected receiver state to verify that the same record cannot be sent in as a duplicate.

The fifth test, reflected record, is meant to verify against attacks where a sent message is bounced immediately back to the origin. This is done by ensuring authentication of source to destination. Effectively, a record that is meant to go from sender to receiver should not be expected to be received by the sender, and thus a mismatch is created and the record is rejected.

The sixth test, of various authentication failures, is meant to test against injections and spoofing. It checks against one entity acting as both communication parties and thus flags the communication and terminates.

Github Repository Link: <https://github.com/DanielCBruni/csce465-hw2-danielbruni>

Daniel Bruni

10/04/2026

Professor Gu

CSCE 465 - 500

Task 4 Security Note:

For these tests, and for the homework as a whole, there are a few key components to understand contextualizing these security measures and threats. First and foremost, the attacker model and non-goals. The attacker can be assumed to have presumably full access to the network, each endpoint, and the transmissions between the entities. As mentioned previously in many of the test cases, the bit flipping, for example, could only occur when the attacker is known to have access to the packets themselves and is able to alter the header or payload in some manner. As such, we consider the attacker model to follow complete access with a wide range of goals. This can include anything from data exfiltration to data alteration. Adopting this model allows us to review a wide variety of security threats and stay well protected across the landscape. Thus this model follows a sort of zero trust, full attacker access concept, where any potential threat can and will be exploited. The non-goals are things likely out of scope for these simple handshakes and checks, so presume any sort of physical spoofing or tampering, especially in the hardware space, are not goals of the attacker. This is a simple monitor, tamper, and exfiltrate style goal. For the reasoning behind using unique identification vectors for CTR, if any vector were to be reused then using the same key would generate an identical keystream. This allows an attacker to XOR two ciphertexts together and completely nullify the keystream, thus exposing the underlying plaintext patterns or relationships. Verifying before decryption is key in this homework because of our attacker model. Assuming the previous model where the attacker has access to the payload along the communication, we must verify the integrity of the header or payload before decryption, otherwise, any sort of integrity failures where something was altered would allow the attacker to successfully complete an injection without checking before decryption. Using independent encryption and MAC keys is also imperative, as if any sort of crossover encryption were to be used, an attacker can study the ciphertext to statistically analyze any sort of patterns or relationships, thus being a step closer to cracking the encryption itself. The main difference between this system and something like AEAD, is that this system cleanly separates confidentiality and integrity, even down to individual metadata. Lastly, a valid record may still contain a dangerous tool call because cryptography and the security principles employed here provide measures to avoid tampering with the payload. But if the payload itself is some command that could be harmful, there is no check that the authorized user is or is not making a mistake. This kind of system simply does not have the kind of contextual perception and critical thinking to solve that problem. Something like an AI agent, for example, would be able to better understand tool calls or commands within the context of the scope given. This being said, simply cryptography is meant only to ensure data integrity and authentication, not data accuracy to that degree or level of depth.

~ 510 words